Recursion and Divide-and-Conquer

Prof. Timothy Sun

Reduction

Reduction: converting a problem ***X*** into a problem ***Y***, where we already have a “black box” algorithm for ***Y***.

Reduction

Reduction: converting a problem ***X*** into a problem ***Y***, where we already have a "black box" algorithm for ***Y***.

Proof of correctness for alg. ***X*** should not depend on details of the black box, though the runtime would.

Reduction - median

Example: Finding the **median** element of an array.

Reduction - median

Example: Finding the **median** element of an array.

68	46	23	72	56	19	30

Reduction - median

Example: Finding the **median** element of an array.

68	46	23	72	56	19	30

Reduction to **sorting**:

- Sort the array, then return ***n***/2-th element.

Reduction - median

68	46	23	72	56	19	30

↓

19	23	30	46	56	68	72

Reduction - median

“Sort the array, then return ***n***/2-th element.”

Reduction - median

“Sort the array, then return ***n***/2-th element.”

- Does it matter if we use, e.g., bubblesort vs. mergesort?

Reduction - median

“Sort the array, then return ***n***/2-th element.”

- Does it matter if we use, e.g., bubblesort vs. mergesort?
  - Correctness: no
  - Runtime: yes

Recursion

Recursive alg. usually consists of two parts:

- Algorithm for a base case
- Reduction to a smaller instance of the same problem

Recursion

Recursive alg. usually consists of two parts:

- Algorithm for a base case
- Reduction to a smaller instance of the same problem

Original instance: ***X***, smaller instance ***Y***. Correctness for ***Y*** by the (strong) inductive hypothesis.

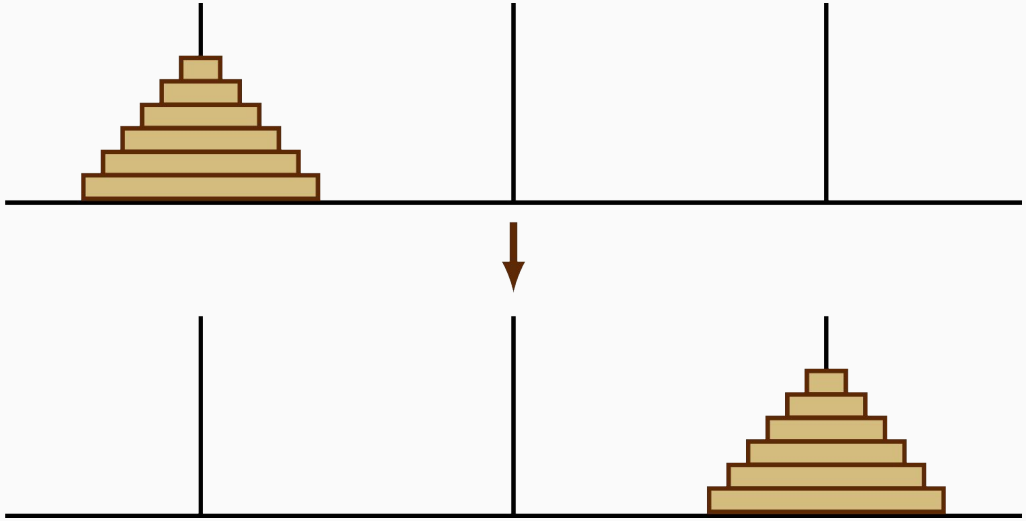
The most important slide in this class

When designing or proving the correctness of a recursive algorithm,

ASSUME RECURSIVE CALLS WORK CORRECTLY

Towers of Hanoi

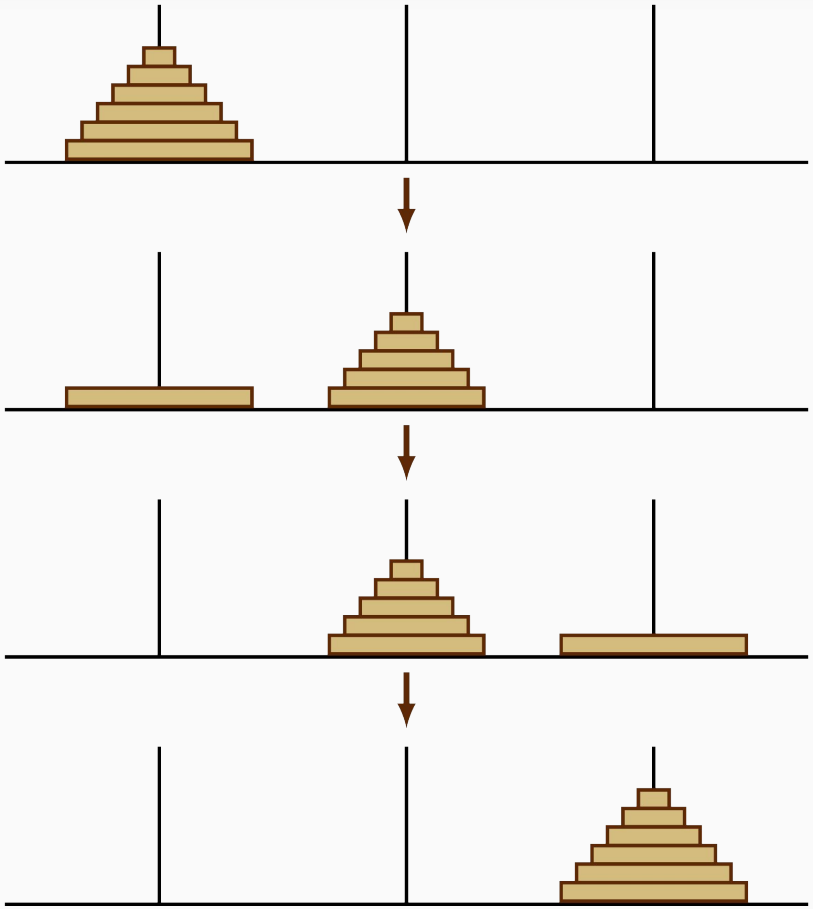
Move all the discs to the right peg, one at a time, without a bigger disc on top of a smaller disc.



Towers of Hanoi

Any solution would consist of three major steps:

1. Move discs 1, …, ***n***-1 to the middle.
2. Move disc ***n*** to the right.
3. Move discs 1, …, ***n***-1 to the right.

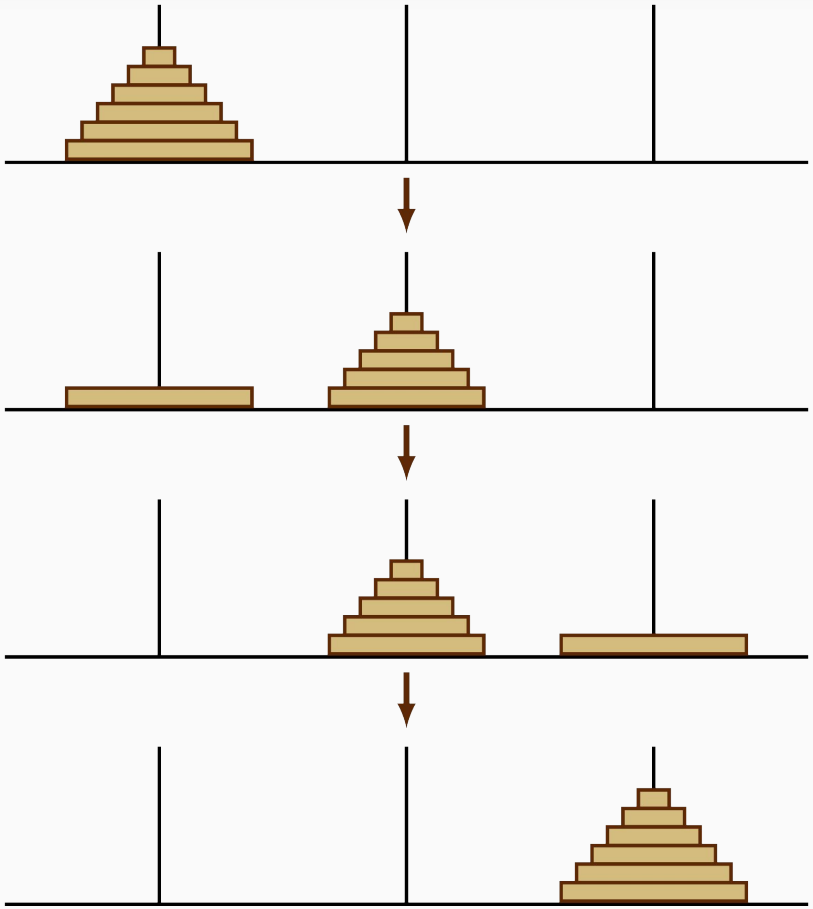


Towers of Hanoi

Any solution would consist of three major steps:

1. Move discs 1, …, ***n***-1 to the middle.
2. Move disc ***n*** to the right.
3. Move discs 1, …, ***n***-1 to the right.

How do we do steps 1 and 3?



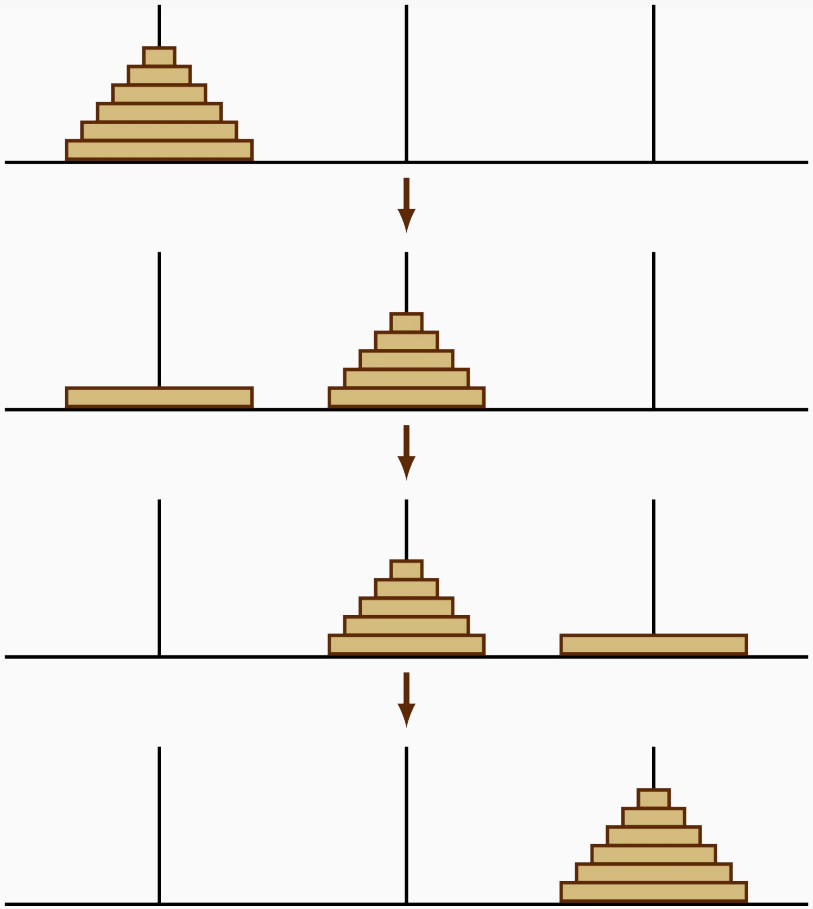
Towers of Hanoi

Any solution would consist of three major steps:

1. Move discs 1, …, ***n***-1 to the middle.
2. Move disc ***n*** to the right.
3. Move discs 1, …, ***n***-1 to the right.

How do we do steps 1 and 3?

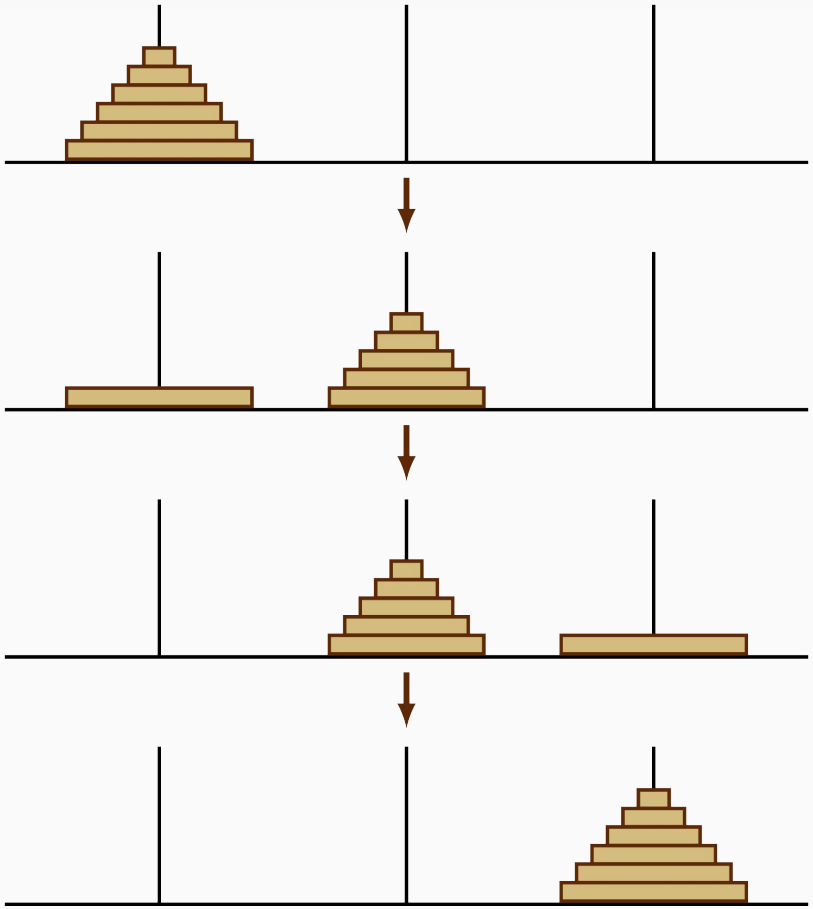
Recursion, recursion, recursion!



Towers of Hanoi

"Move ***n*** discs from 'start' to 'end' with the help of 'other'"

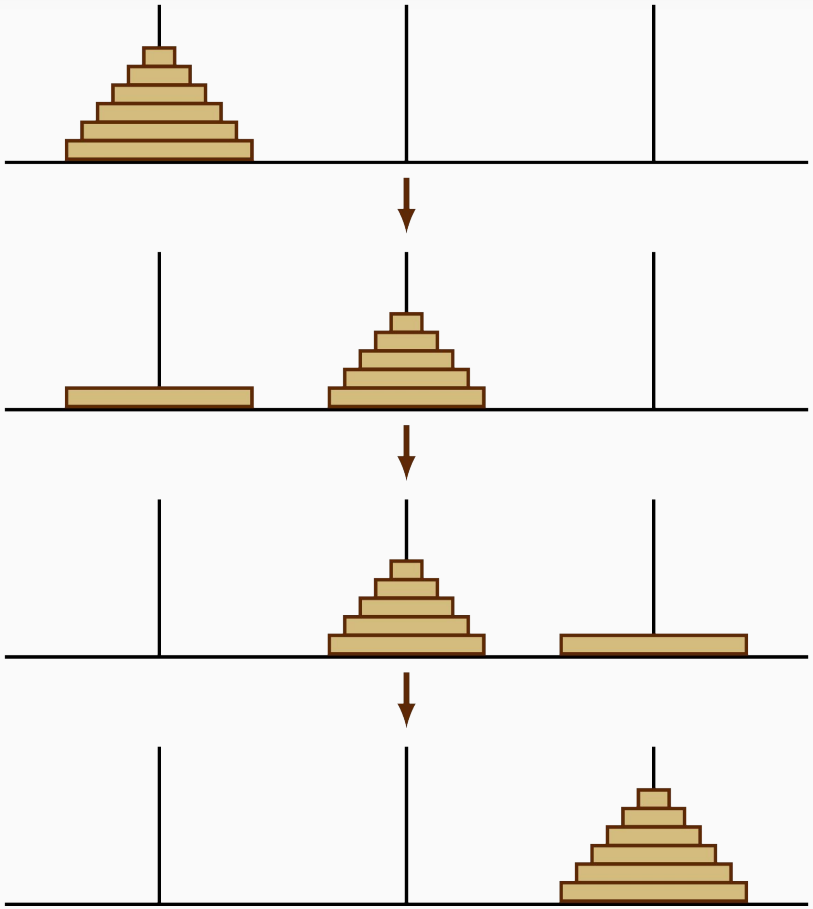
$\textsc{Hanoi}(n, start, end, other)$:
  if $n > 0$
    $\textsc{Hanoi}(n-1, start, other, end)$
    move n: $start \to end$
    $\textsc{Hanoi}(n-1, other, end, start)$



Towers of Hanoi

"Move ***n*** discs from 'start' to 'end' with the help of 'other'"

$\textsc{Hanoi}(n, start, end, other)$:
  if $n > 0$
    $\textsc{Hanoi}(n-1, start, other, end)$
    move n: $start \to end$
    $\textsc{Hanoi}(n-1, other, end, start)$



To prove correctness, no need to unwrap the recursion.

Insertion sort, revisited

$\textsc{InsertionSort}(A[1,\ldots,n])$:

  for $i = 2,\ldots,n$

    $j := i - 1$

    while $j > 0$ and $A[j] > A[j+1]$

      swap $A[j]$ and $A[j+1]$

      $j := j - 1$

iterative

Insertion sort, revisited

$\textsc{InsertionSort}(A[1,\ldots,n])$:
  for $i = 2,\ldots,n$
    $j := i - 1$
    while $j > 0$ and $A[j] > A[j+1]$
      swap $A[j]$ and $A[j+1]$
      $j := j - 1$

iterative

$\textsc{InsertionSort}(A[1,\ldots,n])$:
  if $n > 1$
    $\textsc{InsertionSort}(A[1,\ldots,n-1])$
    $j := i - 1$
    while $j > 0$ and $A[j] > A[j+1]$
      swap $A[j]$ and $A[j+1]$
      $j := j - 1$

recursive

Divide and conquer

Divide and conquer is a recursive technique for algorithm design:

- Reduce to multiple smaller instances of the same problem.
- Solve each of those instances.
- Recombine into the solution of the original problem.

Divide and conquer

Divide and conquer is a recursive technique for algorithm design:

- Reduce to multiple smaller instances of the same problem.
- Solve each of those instances.
- Recombine into the solution of the original problem.

Which sorting algorithms are examples of divide and conquer?

Divide and conquer

Divide and conquer is a recursive technique for algorithm design:

- Reduce to multiple smaller instances of the same problem.
- Solve each of those instances.
- Recombine into the solution of the original problem.

Which sorting algorithms are examples of divide and conquer?

mergesort and ***quicksort***

Two definitions of sortedness

An array $\mathbf{A}[1, \ldots, \boldsymbol{n}]$ is ***sorted*** if:

- For any pair of indices $\boldsymbol{i} < \boldsymbol{j}$, $\mathbf{A}[\boldsymbol{i}] \leq \mathbf{A}[\boldsymbol{j}]$.

Two definitions of sortedness

An array $\mathbf{A}[1, \ldots, \boldsymbol{n}]$ is ***sorted*** if:

- For any pair of indices $\boldsymbol{i} < \boldsymbol{j}$, $\mathbf{A}[\boldsymbol{i}] \leq \mathbf{A}[\boldsymbol{j}]$.

… or equivalently:

- For any pair of consecutive indices $\boldsymbol{i}, \boldsymbol{i}+1$, $\mathbf{A}[\boldsymbol{i}] \leq \mathbf{A}[\boldsymbol{i}+1]$.

Two definitions of sortedness

An array $\mathbf{A}[1, \ldots, \boldsymbol{n}]$ is ***sorted*** if:

- For any pair of indices $\boldsymbol{i} < \boldsymbol{j}$, $\mathbf{A}[\boldsymbol{i}] \leq \mathbf{A}[\boldsymbol{j}]$.

… or equivalently:

- For any pair of consecutive indices $\boldsymbol{i}, \boldsymbol{i}+1$, $\mathbf{A}[\boldsymbol{i}] \leq \mathbf{A}[\boldsymbol{i}+1]$.

(induction, induction, induction…)

Concatenation

Given two arrays **A**[1, … , ***n***], and **B**[1, … , ***m***], their ***concatenation*** **A** ○ **B** is the array you get by sticking them back to back.

When is a concatenation sorted?

Concatenation

Given two arrays **A**[1, … , ***n***], and **B**[1, … , ***m***], their ***concatenation*** **A** ○ **B** is the array you get by sticking them back to back.

When is a concatenation sorted?

When A and B are sorted, and A[*n*] ≤ **B**[1].

Quicksort

$\textsc{QuickSort}(A[1,\ldots,n])$:

  if $n > 1$

    choose a pivot $A[p]$

    $L, M, R := \textsc{Partition}(A, A[p])$

    $L' := \textsc{QuickSort}(L)$

    $R' := \textsc{QuickSort}(R)$

    return $L' \circ M \circ R'$

$\textsc{Partition}(A[1,\ldots,n], x)$:

  initialize lists L, M, R

  for $i = 1, \ldots, n$

    if $A[i] < x$

      $L.\textsc{append}(A[i])$

    else if $A[i] = x$

      $M.\textsc{append}(A[i])$

    else

      $R.\textsc{append}(A[i])$

  return L, M, R

"Partition around the pivot, recursively sort smaller and bigger, concatenate."

Quicksort

$\textsc{QuickSort}(A[1,\ldots,n])$:

  if $n > 1$

    choose a pivot $A[p]$

    $L, M, R := \textsc{Partition}(A, A[p])$

    $L' := \textsc{QuickSort}(L)$

    $R' := \textsc{QuickSort}(R)$

    return $L' \circ M \circ R'$

$\textsc{Partition}(A[1,\ldots,n], x)$:

  initialize lists L, M, R

  for $i = 1, \ldots, n$

    if $A[i] < x$

      $L.\textsc{append}(A[i])$

    else if $A[i] = x$

      $M.\textsc{append}(A[i])$

    else

      $R.\textsc{append}(A[i])$

  return L, M, R

How do we know that the recursive calls "work"? Proof is by induction.

Quicksort

Base case: ***n*** = 0 or 1.

$\textsc{QuickSort}(A[1,\ldots,n])$:

  if $n > 1$

    choose a pivot $A[p]$

    $L, M, R := \textsc{Partition}(A, A[p])$

    $L' := \textsc{QuickSort}(L)$

    $R' := \textsc{QuickSort}(R)$

    return $L' \circ M \circ R'$

Quicksort

Base case: ***n*** = 0 or 1.

- Correctly does nothing.

Inductive step: ***if*** quicksort is correct for array sizes 0, …, ***k***, would it be correct for array size ***k***+1?

$$
\begin{aligned}
&\textsc{QuickSort}(A[1,\ldots,n]):\\
&\quad \textbf{if } n > 1\\
&\qquad \text{choose a pivot } A[p]\\
&\qquad L, M, R := \textsc{Partition}(A, A[p])\\
&\qquad L' := \textsc{QuickSort}(L)\\
&\qquad R' := \textsc{QuickSort}(R)\\
&\qquad \textbf{return } L' \circ M \circ R'
\end{aligned}
$$

Quicksort

Base case: ***n*** = 0 or 1.

- Correctly does nothing.

Inductive step: ***if*** quicksort is correct for array sizes 0, …, ***k***, would it be correct for array size ***k***+1?

- IH + concatenation proof

$$
\begin{aligned}
&\textsc{QuickSort}(A[1,\ldots,n]):\\
&\quad \textbf{if } n > 1\\
&\qquad \text{choose a pivot } A[p]\\
&\qquad L, M, R := \textsc{Partition}(A, A[p])\\
&\qquad L' := \textsc{QuickSort}(L)\\
&\qquad R' := \textsc{QuickSort}(R)\\
&\qquad \textbf{return } L' \circ M \circ R'
\end{aligned}
$$

Quicksort

Base case: ***n*** = 0 or 1.

- Correctly does nothing.

Inductive step: ***if*** quicksort is correct for array sizes 0, …, ***k***, would it be correct for array size ***k***+1?

- IH + concatenation proof

$$
\begin{aligned}
&\textsc{QuickSort}(A[1,\ldots,n]):\\
&\quad \textbf{if } n > 1\\
&\qquad \text{choose a pivot } A[p]\\
&\qquad L, M, R := \textsc{Partition}(A, A[p])\\
&\qquad L' := \textsc{QuickSort}(L)\\
&\qquad R' := \textsc{QuickSort}(R)\\
&\qquad \textbf{return } L' \circ M \circ R'
\end{aligned}
$$

By the principle of induction, quicksort is correct for all ***n*** ≥ 0.

Mergesort

$\textsc{MergeSort}(A[1,\ldots,n])$:
  if $n > 1$
    $m := \lfloor n/2 \rfloor$
    $\textsc{MergeSort}(A[1,\ldots,m])$
    $\textsc{MergeSort}(A[m+1,\ldots,n])$
    $\textsc{Merge}(A,m)$

$\textsc{Merge}(A[1,\ldots,n],m)$:
  initialize $B[1,\ldots,n]$
  $i := 1, j := m+1$
  for $k = 1,\ldots,n$
    $B[k] :=$ // smallest
  for $k = 1,\ldots,n$
    $A[k] := B[k]$

Similar proof of correctness for mergesort.

Why does this work?

$\textsc{SlowSort}(A[1,\ldots,n])$:

  if $n > 1$

    $h := \lfloor n/2 \rfloor$

    $\textsc{SlowSort}(A[1,\ldots,h])$

    $\textsc{SlowSort}(A[h+1,\ldots,n])$

    if $A[h] > A[n]$

      swap $A[h]$ and $A[n]$

    $\textsc{SlowSort}(A[1,\ldots,n-1])$

Impossible to imagine the recursion!

Why does this work?

$\textsc{SlowSort}(A[1,\ldots,n])$:

Base case: ***n*** = 0, 1 like before

$\quad$ **if** $n > 1$

$\quad\quad h := \lfloor n/2 \rfloor$

$\quad\quad \textsc{SlowSort}(A[1,\ldots,h])$

$\quad\quad \textsc{SlowSort}(A[h+1,\ldots,n])$

$\quad\quad$ **if** $A[h] > A[n]$

$\quad\quad\quad$ swap $A[h]$ and $A[n]$

$\quad\quad \textsc{SlowSort}(A[1,\ldots,n-1])$

Why does this work?

$\textsc{SlowSort}(A[1,\ldots,n])$:
  if $n > 1$
    $h := \lfloor n/2 \rfloor$
    $\textsc{SlowSort}(A[1,\ldots,h])$
    $\textsc{SlowSort}(A[h+1,\ldots,n])$
    if $A[h] > A[n]$
      swap $A[h]$ and $A[n]$
    $\textsc{SlowSort}(A[1,\ldots,n-1])$

Base case: ***n*** = 0, 1 like before

Inductive step:

- Find and place largest

Why does this work?

$\textsc{SlowSort}(A[1,\ldots,n])$:
  if $n > 1$
    $h := \lfloor n/2 \rfloor$
    $\textsc{SlowSort}(A[1,\ldots,h])$
    $\textsc{SlowSort}(A[h+1,\ldots,n])$
    if $A[h] > A[n]$
      swap $A[h]$ and $A[n]$
    $\textsc{SlowSort}(A[1,\ldots,n-1])$

Base case: ***n*** = 0, 1 like before

Inductive step:

- Find and place largest
- Sort the rest

Maximum subarray

Given an array of integers

-54	-83	-47	93	-62	60	9	77	70	-85	-3	58	-1	-15	-58	79

What's the subarray whose sum is the biggest?

Maximum subarray

Given an array of integers

-54	-83	-47	93	-62	60	9	77	70	-85	-3	58	-1	-15	-58	79

What's the subarray whose sum is the biggest? 216?

Maximum subarray

Given an array of integers

-54	-83	-47	93	-62	60	9	77	70	-85	-3	58	-1	-15	-58	79

What's the subarray whose sum is the biggest? **247**, best possible

Maximum subarray

-54	-83	-47	93	-62	60	9	77	70	-85	-3	58	-1	-15	-58	79

Brute force: calculate each subarray

Maximum subarray

-54	-83	-47	93	-62	60	9	77	70	-85	-3	58	-1	-15	-58	79

Brute force: calculate each subarray

```
MaxSubBF1(A[1,...,n]):
    bestSum := 0
    for i = 1,...,n
        for j = i,...,n
            sum := 0
            for k = i,...,j
                sum := sum + A[k]
            if sum > bestSum
                bestSum := sum
    return bestSum
```

Maximum subarray

-54	-83	-47	93	-62	60	9	77	70	-85	-3	58	-1	-15	-58	79

Brute force: calculate each subarray

Runtime: $\Theta(n^3)$!

```
MaxSubBF1(A[1,...,n]):
    bestSum := 0
    for i = 1,...,n
        for j = i,...,n
            sum := 0
            for k = i,...,j
                sum := sum + A[k]
            if sum > bestSum
                bestSum := sum
    return bestSum
```

Maximum subarray

-54	-83	-47	93	-62	60	9	77	70	-85	-3	58	-1	-15	-58	79

Brute force: calculate each subarray

Runtime: $\Theta(n^3)$, due to repeated work

```
MaxSubBF1(A[1,...,n]):
    bestSum := 0
    for i = 1,...,n
        for j = i,...,n
            sum := 0
            for k = i,...,j
                sum := sum + A[k]
            if sum > bestSum
                bestSum := sum
    return bestSum
```

Maximum “prefix”

-54	-83	-47	93	-62	60	9	77	70	-85	-3	58	-1	-15	-58	79

How to develop a $\Theta(\boldsymbol{n})$ algorithm?

Maximum “prefix”

-54	-83	-47	93	-62	60	9	77	70	-85	-3	58	-1	-15	-58	79

How to develop a Θ(***n***) algorithm? **Cumulative sums**.

$\textsc{MaxPrefix}(A[1,\ldots,n])$:
    $bSum := 0$
    $cSum := 0$
    for $i = 1,\ldots,n$
        $cSum := cSum + A[i]$
        if $cSum > bSum$
            $bSum := cSum$
    return $bSum$

Maximum subarray

```
MaxPrefix(A[1,...,n]):
    bSum := 0
    cSum := 0
    for i = 1,...,n
        cSum := cSum + A[i]
        if cSum > bSum
            bSum := cSum
    return bSum
```

```
MaxSubBF2(A[1,...,n]):
    bestSum := 0
    for i = 1,...,n
        bestAtI := MaxPrefix(A[i,...,n])
        if bestAtI > bestSum
            bestSum := bestAtI
    return bestSum
```

Maximum subarray

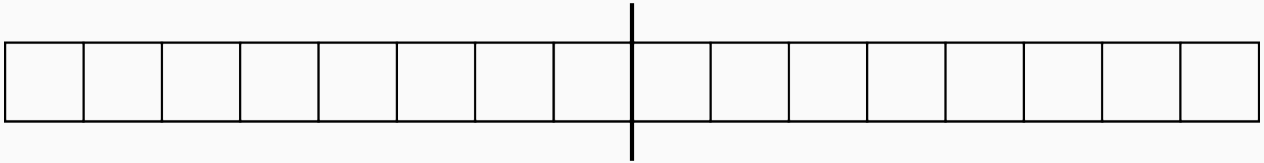
```
MaxPrefix(A[1,...,n]):
    bSum := 0
    cSum := 0
    for i = 1,...,n
        cSum := cSum + A[i]
        if cSum > bSum
            bSum := cSum
    return bSum
```

```
MaxSubBF2(A[1,...,n]):
    bestSum := 0
    for i = 1,...,n
        bestAtI := MaxPrefix(A[i,...,n])
        if bestAtI > bestSum
            bestSum := bestAtI
    return bestSum
```

Runtime: $\Theta(\boldsymbol{n}^2)$

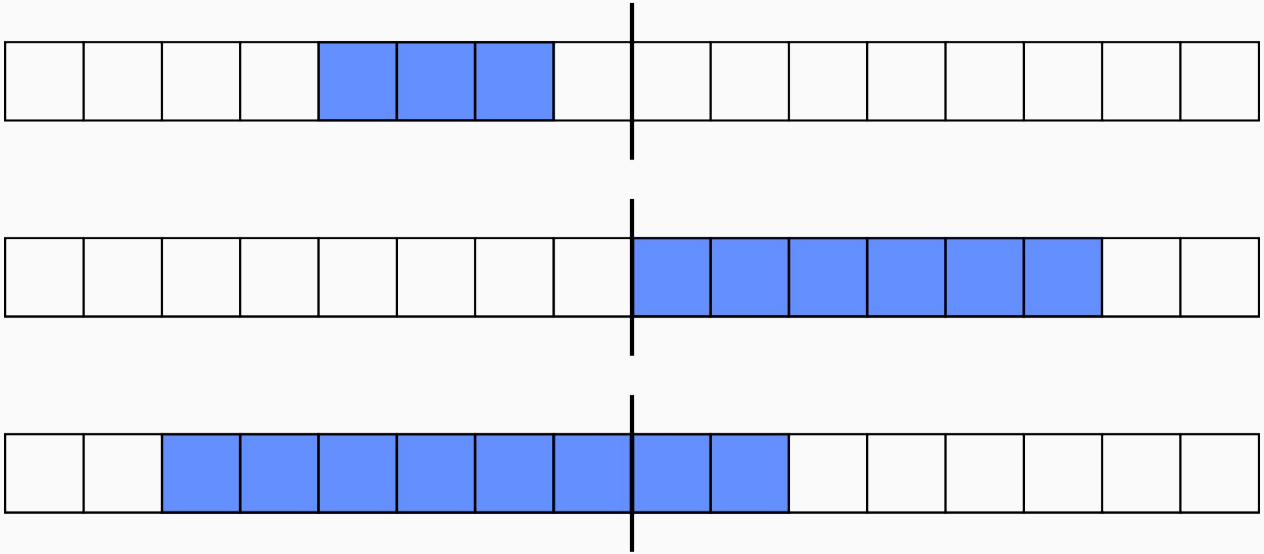
Maximum subarray

A ***divide and conquer*** approach: splitting the array in half



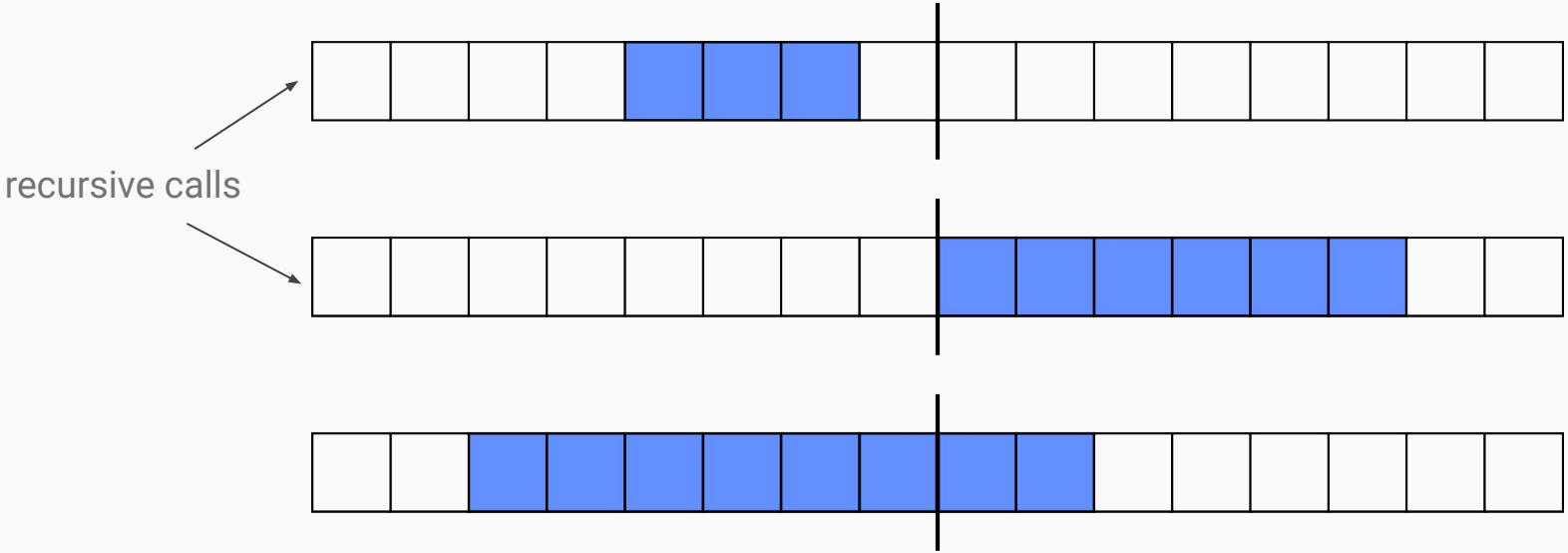
Maximum subarray

A ***divide and conquer*** approach: splitting the array in half

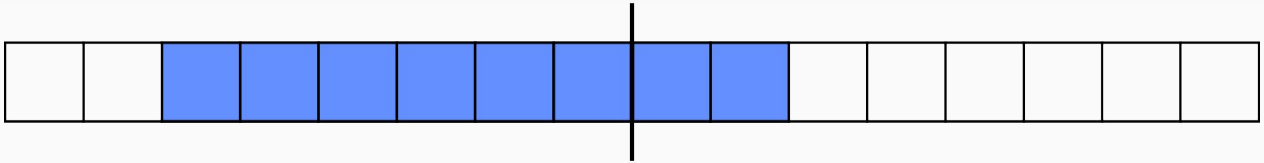


Maximum subarray

A ***divide and conquer*** approach: splitting the array in half

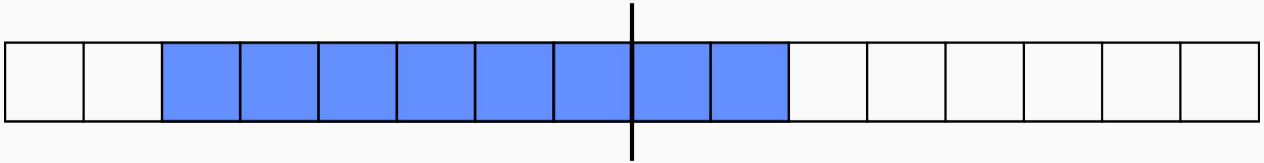


Maximum subarray



What’s the best way to solve this subproblem?

Maximum subarray

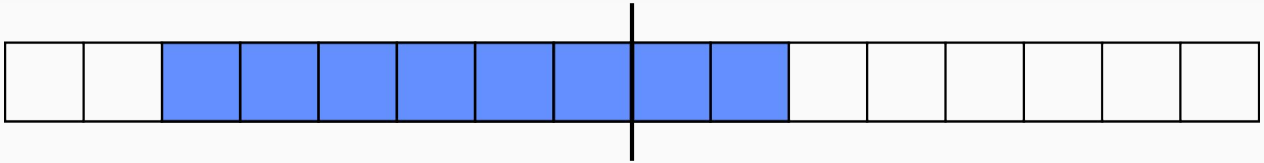


What's the best way to solve this subproblem?

Observation: best solution = best left half + best right half

$$h := \lfloor n/2 \rfloor$$
$$bestFirstHalf := \textsc{MaxPrefix}(A[h, \ldots, 1])$$
$$bestSecondHalf := \textsc{MaxPrefix}(A[h+1, \ldots, n])$$

Maximum subarray



$\textsc{MaxSubDnC}(A[1,\ldots,n])$:

$\quad$ **if** $n = 0$: **return** 0

$\quad$ $h = \lfloor n/2 \rfloor$

$\quad$ $recL := \textsc{MaxSubDnC}(A[1,\ldots,h])$

$\quad$ $recR := \textsc{MaxSubDnC}(A[h+1,\ldots,n])$

$\quad$ $bestMid := \textsc{MaxPrefix}(A[h,\ldots,1]) + \textsc{MaxPrefix}(A[h+1,\ldots,n])$

$\quad$ **return** $\max(recL, recR, bestMid)$

Maximum subarray

$\textsc{MaxSubDnC}(A[1,\ldots,n])$:

if $n = 0$: **return** 0

$h = \lfloor n/2 \rfloor$

$recL := \textsc{MaxSubDnC}(A[1,\ldots,h])$

$recR := \textsc{MaxSubDnC}(A[h+1,\ldots,n])$

$bestMid := \textsc{MaxPrefix}(A[h,\ldots,1]) + \textsc{MaxPrefix}(A[h+1,\ldots,n])$

return $\max(recL, recR, bestMid)$

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$$

Order statistics

For quicksort, guarantee a good pivot, if **median** chosen every time.

Order statistics

For quicksort, guarantee a good pivot, if **median** chosen every time.

The more general problem:

> "Given an unsorted array, find the ***k***-th order statistic (***k***-th smallest elt.)"

Order statistics

For quicksort, guarantee a good pivot, if **median** chosen every time.

The more general problem:

> "Given an unsorted array, find the ***k***-th order statistic (***k***-th smallest elt.)"

Median: ***n***/2-th order statistic (slightly different from median in stats.)

Simple approach

Sort first, pick out ***k***-th element. $\Theta(\boldsymbol{n} \log \boldsymbol{n})$

Simple approach

Sort first, pick out ***k***-th element. $\Theta(\boldsymbol{n} \log \boldsymbol{n})$

Can we do better?

Partition subroutine

Partitioning around the pivot gives us some information in O(***n***) time.

```
PARTITION(A[1,...,n], x):
    initialize empty lists L, M, R
    for i = 1,...,n
        if A[i] < x: L.APPEND(A[i])
        else if A[i] = x: M.APPEND(A[i])
        else: R.APPEND(A[i])
    return L, M, R
```

```
PARTITION(A[1..n], p):
    swap A[p] ↔ A[n]
    ℓ ← 0                    ⟨⟨#items < pivot⟩⟩
    for i ← 1 to n − 1
        if A[i] < A[n]
            ℓ ← ℓ + 1
            swap A[ℓ] ↔ A[i]
    swap A[n] ↔ A[ℓ + 1]
    return ℓ + 1
```

Partition subroutine

Partitioning around the pivot gives us some information in O(***n***) time.

60	77	35	42	94	21	53	16	88

Partition subroutine

Partitioning around the pivot gives us some information in O(***n***) time.

60	77	35	42	94	21	53	16	88

L:

35	42	21	53	16

M:

60

R:

77	94	88

Partition subroutine

Partitioning around the pivot gives us some information in O(***n***) time.

60	77	35	42	94	21	53	16	88

L:

35	42	21	53	16

M:

60

R:

77	94	88

Based on size of ***L***, we can figure out the “rank” of the pivot.

Quickselect

Like quicksort in that:

- it partitions the elements

```
QuickSelect(A[1, ..., n], k):
    if n = 1
        return A[1]
    else
        choose a pivot p from A
        L, M, R := Partition(A, p)
        if k ≤ |L|
            return QuickSelect(L, k)
        else if k ≤ |L| + |M|
            return p
        else
            return QuickSelect(R, k − |L| − |M|)
```

Quickselect

$$\textsc{QuickSelect}\left(\begin{array}{|c|c|c|c|c|c|c|c|c|}\hline 60 & 77 & 35 & 42 & 94 & 21 & 53 & 16 & 88 \\ \hline\end{array}, k = 4\right)$$

L:

35	42	21	53	16

M:

60

R:

77	94	88

Quickselect

$$\textsc{QuickSelect}\left(\boxed{35}\,\boxed{42}\,\boxed{21}\,\boxed{53}\,\boxed{16}, k = 4\right)$$

L: $\boxed{21}\,\boxed{16}$

M: $\boxed{35}$

R: $\boxed{42}\,\boxed{53}$

Quickselect

$$\textsc{QuickSelect}(\boxed{42}\,\boxed{53}, k = 1)$$

$$L: \boxed{}$$

$$M: \boxed{42}$$

$$R: \boxed{53}$$

Quickselect

Like quicksort in that:

- it partitions the elements
- **runtime depends on pivot quality**

```
QuickSelect(A[1, ..., n], k):
    if n = 1
        return A[1]
    else
        choose a pivot p from A
        L, M, R := Partition(A, p)
        if k ≤ |L|
            return QuickSelect(L, k)
        else if k ≤ |L| + |M|
            return p
        else
            return QuickSelect(R, k − |L| − |M|)
```

Quickselect

In worst case, might need to recurse on bigger half:

$$T(n) = \max(T(\ell), T(r)) + \Theta(n)$$

Quickselect

In worst case, might need to recurse on bigger half:

$$
\begin{aligned}
T(n) &= \max(T(\ell), T(r)) + \Theta(n) \\
&= T(n-1) + \Theta(n) \\
&= \Theta(n^2)
\end{aligned}
$$

Quickselect

Let’s say pivot is always the median.

Quickselect

Let's say pivot is always the median.

$$T(n) = T(n/2) + \Theta(n)$$

Quickselect

Let's say pivot is always the median.

$$T(n) = T(n/2) + \Theta(n) = \Theta(n)$$

Quickselect

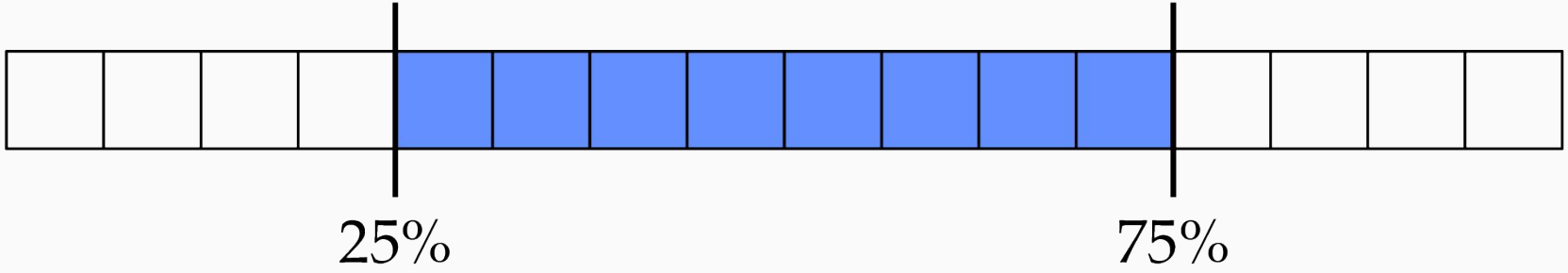
Let's say pivot is always the median.

$$T(n) = T(n/2) + \Theta(n) = \Theta(n)$$

But isn't this circular??

A simple pivot selection

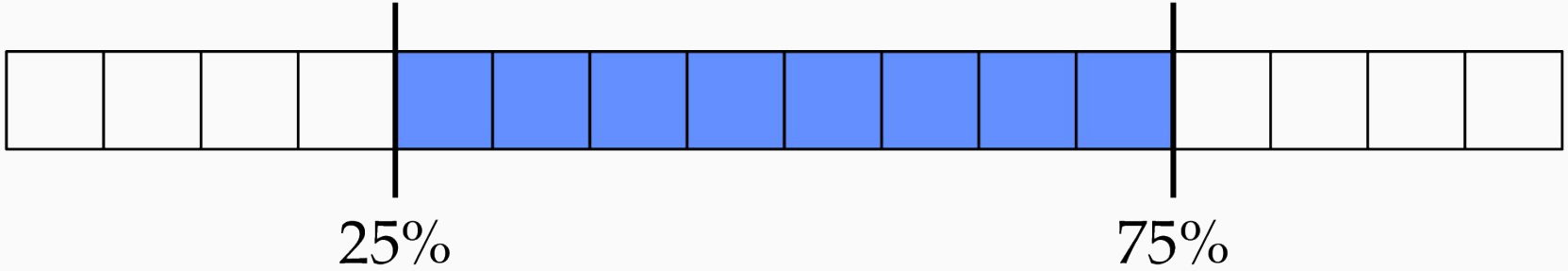
Standard choice: choose a **random** pivot element.



Probability of landing in shaded region?

A simple pivot selection

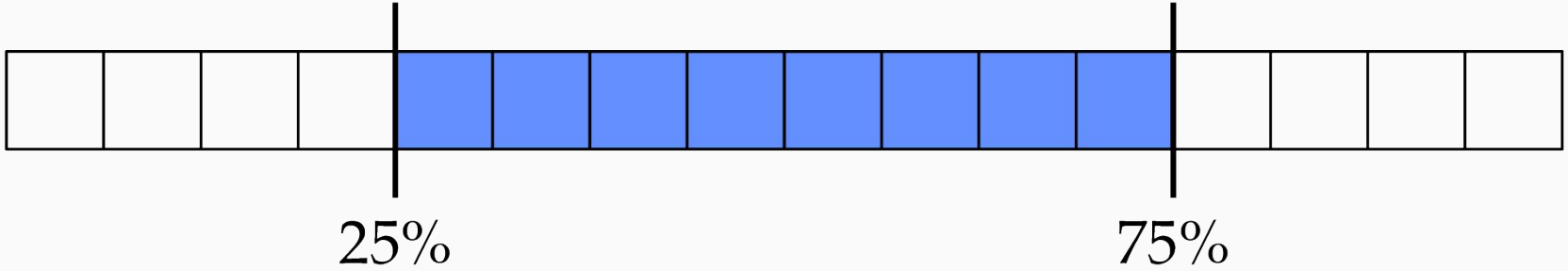
Standard choice: choose a **random** pivot element.



Probability of landing in shaded region? **1/2**

A simple pivot selection

Standard choice: choose a **random** pivot element.

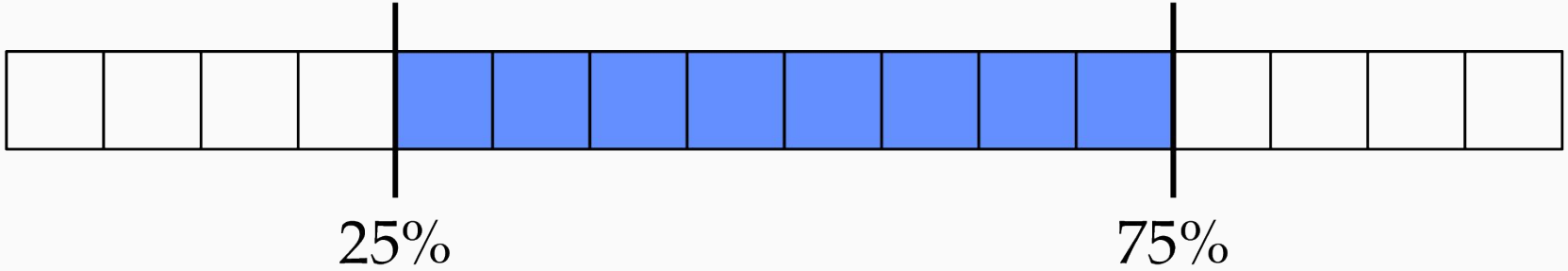


Probability of landing in shaded region? **1/2**

How many tries on average?

A simple pivot selection

Standard choice: choose a **random** pivot element.



Probability of landing in shaded region? **1/2**

How many tries on average? **2** (geometric distribution, ***p*** = 1/2)